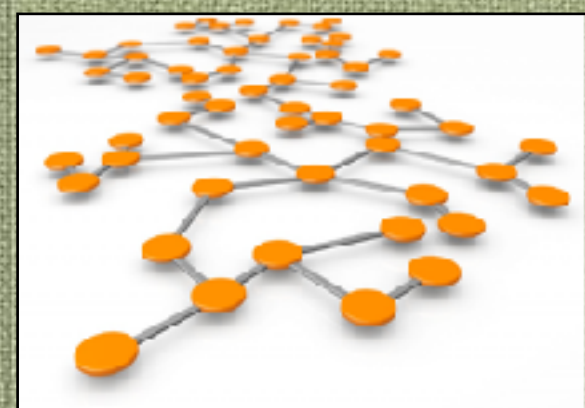




Sistemas Distribuídos

Professor: Rodrigo da Rosa Righi

Contato: rrrighi@unisinus.br

Aula: 15

Dia/Horário: Sexta-Feira, 19:30 - 22:23

Agenda

- * Tolerância a Falhas
- * Introdução
- * Grupos de Processos
- * Comunicação Cliente-Servidor Confiável
- * Recuperação (Recovery)
- * Cloud Computing

Introdução

- ❖ Antes de entender o papel de tolerância a falhas em sistemas distribuídos, nós precisamos verificar os itens necessários para a um sistema distribuído ser tolerante a falhas.
 - ❖ A tolerância a falhas está ligada com **Dependable systems**. Dependability é o termo que cobre um número de requerimentos úteis para sistemas distribuídos, tais como:
 - ❖ Availability - Disponibilidade
 - ❖ Reliability - Confiabilidade
 - ❖ Safety - Segurança
 - ❖ Maintainability - Facilidade de Manutenção
- ❖ **Disponibilidade:** O sistema está pronto para ser usado imediatamente. Um sistema com Alta disponibilidade é aquele que se mantém funcionando em um dado instante de tempo.
- ❖ **Confiabilidade:** Propriedade que considera que o sistema execute suas funções continuamente sem falhas. Contrário do termo Disponibilidade, Confiabilidade é definido em termos de intervalo de tempo ao invés de um instante no tempo. Um sistema com alta confiabilidade é aquele que continuará a trabalhar sem interrupções durante um longo período de tempo.
 - ❖ Importante: Se um sistema cai em um determinado milisegundo a cada hora, ele possui uma disponibilidade 99.99%, mas é altamente não confiável. Similarmente, um sistema que nunca cai mas deve ser desligado duas semanas a cada Agosto, o sistema como um todo tem alta confiabilidade mas 96% de disponibilidade.

Introdução

$$\textit{Disponibilidade (\%)} = \frac{\textit{horas disponíveis}}{\textit{horas planejadas}} \times 100\%$$

$$\textit{Confiabilidade} = \frac{\textit{horas disponíveis}}{\textit{número de danos}}$$

Introdução

- ❖ **Segurança:** Se refere a situações onde o sistema temporariamente falha em operar corretamente. Por exemplo, muitas tarefas de controle de sistema como aquelas que controlam bases nucleares ou enviam pessoas ao espaço, devem apresentar alto grau de Segurança. Se o sistema de controle temporariamente falhar por um breve momento, o efeito pode ser desastroso.
 - ❖ É muito difícil desenvolver sistemas 100% Safety.
- ❖ **Facilidade de Manutenção:** Se refere a fato de o quanto é fácil reparar um sistema com falhas. Um sistema com alto grau de Maintainability deve apresentar também um alto grau de Availability, especialmente quando falhas podem ser detectadas e reparadas automaticamente.
- ❖ Outros termos:
 - ❖ **Fail system** - Sistema não cumpre com as suas promessas. Por exemplo: um sistema com vários serviços é dito como Failed quando um deles não é completamente provido.
 - ❖ **Erro** - Parte do estado de um sistema que pode levar a uma falha. Por exemplo: ao transportar pacotes pela rede, é esperado que alguns pacotes estejam corrompidos quando eles chegam no receptor. Corrompidos nesse contexto indica um bit errado.
 - ❖ A causa de um erro é chamada **falha (fault)**: Importância de verificar o que determinou a ocorrência de um erro. Por exemplo: um meio de transmissão ruim pode facilmente acarretar em pacotes corrompidos.

Introdução

Conceito	Tradução Comum	Definição	Exemplo
Fault	Defeito / Falta	A causa subjacente ou anomalia (defeito no código, falha de hardware, erro de projeto) que pode, potencialmente, levar a um estado incorreto.	Um <i>bug</i> em uma linha de código; um <i>chip</i> de memória defeituoso.
Error	Erro	O estado incorreto interno que é criado quando o Fault é ativado. É o desvio do estado correto do sistema.	Uma variável na memória assume um valor incorreto; um ponteiro inválido.
Failure	Falha / Fracasso	A manifestação externa do erro, onde o sistema não consegue cumprir seu serviço conforme especificado pelo usuário ou sistema externo.	O sistema trava (<i>crash</i>); um servidor <i>web</i> para de responder a requisições; <i>output</i> incorreto.
Fail-Stop System	Sistema de Parada por Falha	Um sistema projetado para, ao detectar um erro, parar imediatamente de produzir qualquer saída, garantindo que não propagará dados inconsistentes.	Um nó em um <i>cluster</i> de computadores que se desliga (entra em <i>fail-stop</i>) ao detectar uma corrupção de dados.

Introdução

O Ciclo: **Fault** → **Error** → **Failure**

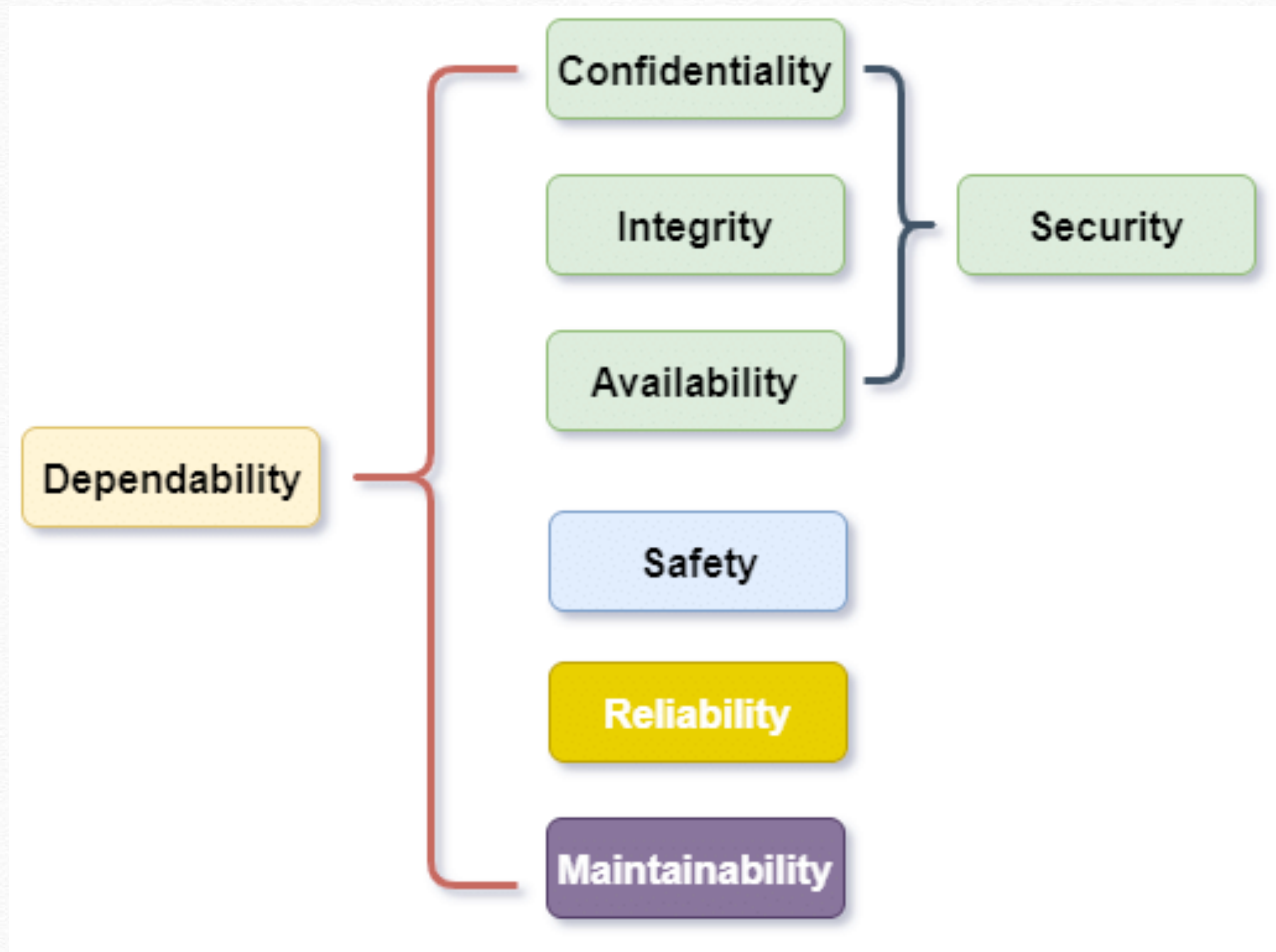
Esses conceitos estão ligados por uma cadeia de eventos: Um **Fault** (defeito latente) → pode ser **ativado** → causando um **Error** (estado incorreto interno) → que se **propaga** → e leva a um **Failure** (falha do sistema em cumprir sua função).

Conceito Adicional: Fail-Stop System

Um **Fail-Stop System** é um tipo de sistema tolerante a falhas que, na presença de um *Fault*, é projetado para **parar imediatamente e não produzir mais nenhuma saída (output)**.

- **Conceito:** Garante que, se uma falha ocorrer, o sistema simplesmente **para**, mas o faz de forma consistente, evitando que dados incorretos sejam propagados para outros sistemas ou usuários.
- **Exemplo:** Em um sistema distribuído, se um nó detecta uma falha interna, ele para de responder e não envia mensagens, permitindo que os nós restantes detectem sua ausência e assumam suas tarefas, garantindo a consistência e prevenindo um **Byzantine Failure** (onde o sistema entra em um estado inconsistente e produz um *output* errado).

Introdução



Introdução

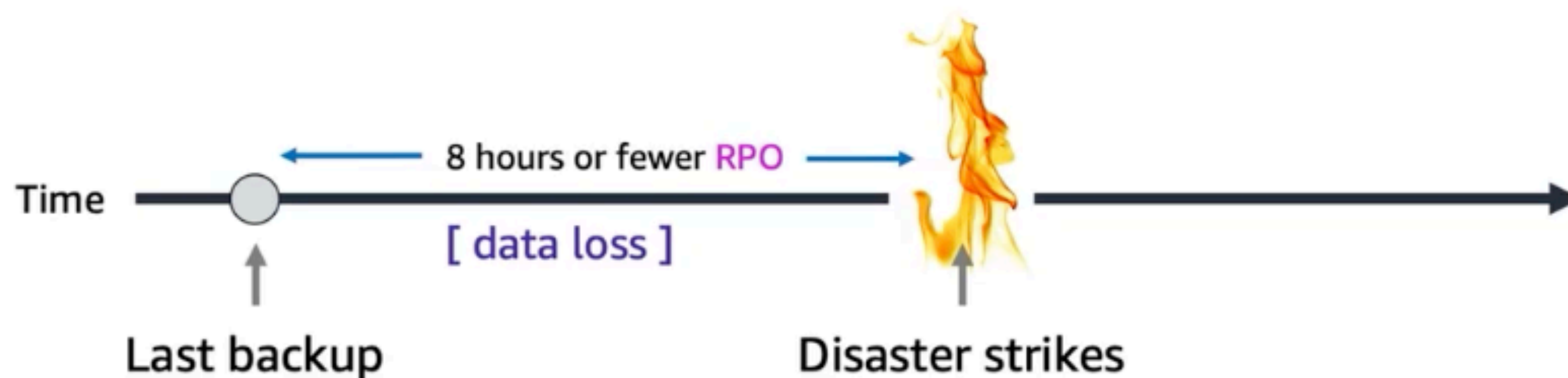
Recovery point objective (RPO)



Recovery point objective (RPO) is the maximum acceptable amount of data loss, measured in time.

How often must your data be backed up?

Example RPO: *The business can recover from losing (at most) the last 8 hours of data.*



Introdução

- ❖ Falhas (Faults) são geralmente classificadas como: transientes, intermitentes e permanentes.
- ❖ **Falhas transientes** ocorrem uma vez e desaparecem.
 - ❖ Importante: Se a operação é repetida, a falha não aparece. Por exemplo: conjunto de pássaros perto de transmissor de microondas faz com que vários bits sejam perdidos. Uma retransmissão resolve o problema.
- ❖ **Falhas intermitentes** ocorrem aleatoriamente e podem reaparecer. Elas agravam o problema, porque elas são de difícil diagnóstico.
- ❖ **Falhas permanentes** são aquelas que continuam a existir até que o componente com falha seja reparado. Exemplo: Bugs de software e crash na cabeça de um disco.

Introdução

Tipo de Fault	Característica	Propriedade de Repetição	Novo Exemplo
Transiente	Ocorre uma única vez e desaparece.	Não reaparece se a operação for repetida.	Uma flutuação momentânea na tensão da rede elétrica causa um único erro de leitura em um sensor. Uma nova leitura subsequente é bem-sucedida.
Intermitente	Ocorre de forma aleatória e pode reaparecer em momentos imprevisíveis.	Reaparece aleatoriamente, dificultando o diagnóstico.	Uma conexão de rede solta em um servidor que causa quedas de conexão esporádicas, dependendo da vibração do ambiente ou do movimento de pessoas próximas.
Permanente	Continua a existir até que o componente defeituoso seja reparado ou substituído.	Persiste continuamente (100% de chance de ocorrência).	Uma linha de código que nunca verifica o valor nulo de um ponteiro (<i>Null Pointer Exception</i>), causando o travamento do programa toda vez que essa seção de código é executada.

Introdução

Type of failure	Description
Crash failure	A server halts, but is working correctly until it halts
Omission failure <i>Receive omission</i> <i>Send omission</i>	A server fails to respond to incoming requests A server fails to receive incoming messages A server fails to send messages
Timing failure	A server's response lies outside the specified time interval
Response failure <i>Value failure</i> <i>State transition failure</i>	A server's response is incorrect The value of the response is wrong The server deviates from the correct flow of control
Arbitrary failure	A server may produce arbitrary responses at arbitrary times

Figura 1. Diferentes tipos de falhas

Introdução

- ❖ **Crash** ocorre quando um servidor prematuramente congela, mas estava trabalhando corretamente até esse ponto. Exemplo: sistemas operacionais que congelam e a única solução é um reboot.
- ❖ **Falhas de omissão** ocorrem quando um servidor falha ao responder a uma requisição. Várias coisas poderiam dar errado. No caso de falha por omissão na recepção, o servidor pode ter nunca capturado a requisição. A falha por omissão no envio ocorre quando o servidor faz seu trabalho, mas algo falha na transmissão da resposta. Isso pode ocorrer quando há buffer overflow enquanto o servidor não estava preparado para essa situação. O servidor deve estar preparado para uma retransmissão do cliente.
- ❖ **Falhas de Tempo** (Timing Failures) ocorrem quando a resposta ocorre fora de um intervalo real-time especificado. Mandar dados prematuramente pode ocasionar nesse tipo de erro, pois pode não haver buffer alocado no receptor. Entretanto, o mais comum é que servidores respondam muito tarde.
- ❖ **Falha de Resposta** (Response Failure) representa um sério tipo de falha, na qual a resposta do servidor está simplesmente incorreta.
 - ❖ Uma falha de valor ocorre quando o servidor fornece uma resposta errada para uma requisição. Por exemplo: serviço de busca que sempre retorna páginas Web não relacionadas com as palavras chaves dadas como entrada.
 - ❖ Uma falha por estado de transição acontece quando o servidor reage inesperadamente a uma requisição de entrada. Por exemplo: se o servidor recebe uma mensagem que ele não reconhece, a falha por estado de transição ocorre caso nenhuma medida for aplicada para essa mensagem de entrada.

Introdução

- ❖ **Falhas arbitrárias** também são conhecidas como Bizantinas. Clientes devem estar preparados para o pior. Por exemplo: um servidor produzir uma saída que ele nunca fez antes, mas que não pode ser detectada como sendo incorreta.
- ❖ Relações:
 - ❖ Falhas arbitrárias são relacionadas com falhas de crash. A definição de crash da lâmina anterior também é conhecida como **fail-stop failure**. De fato, Um servidor fail-stop irá parar de produzir saída de modo que seu desligamento **possa ser detectado por outros processos**.
 - ❖ Entretanto, na vida real, servidores param sem avisar sobre falhas de crash, e assim, não há o aviso que eles irão parar. O outro processo é que deve decidir se o servidor parou prematuramente. Nesses sistemas **fail-silent**, outros processos podem incorretamente concluir que um servidor parou. O servidor pode estar devagar inesperadamente.
 - ❖ Quando um servidor está produzindo saídas aleatórias mas elas são reconhecidas como lixo pelos demais processos. O servidor está exibindo falhas arbitrárias, mas de modo benigno. Essas falhas são também referenciadas como **fail-safe**.

Two flavours of FAULTS:

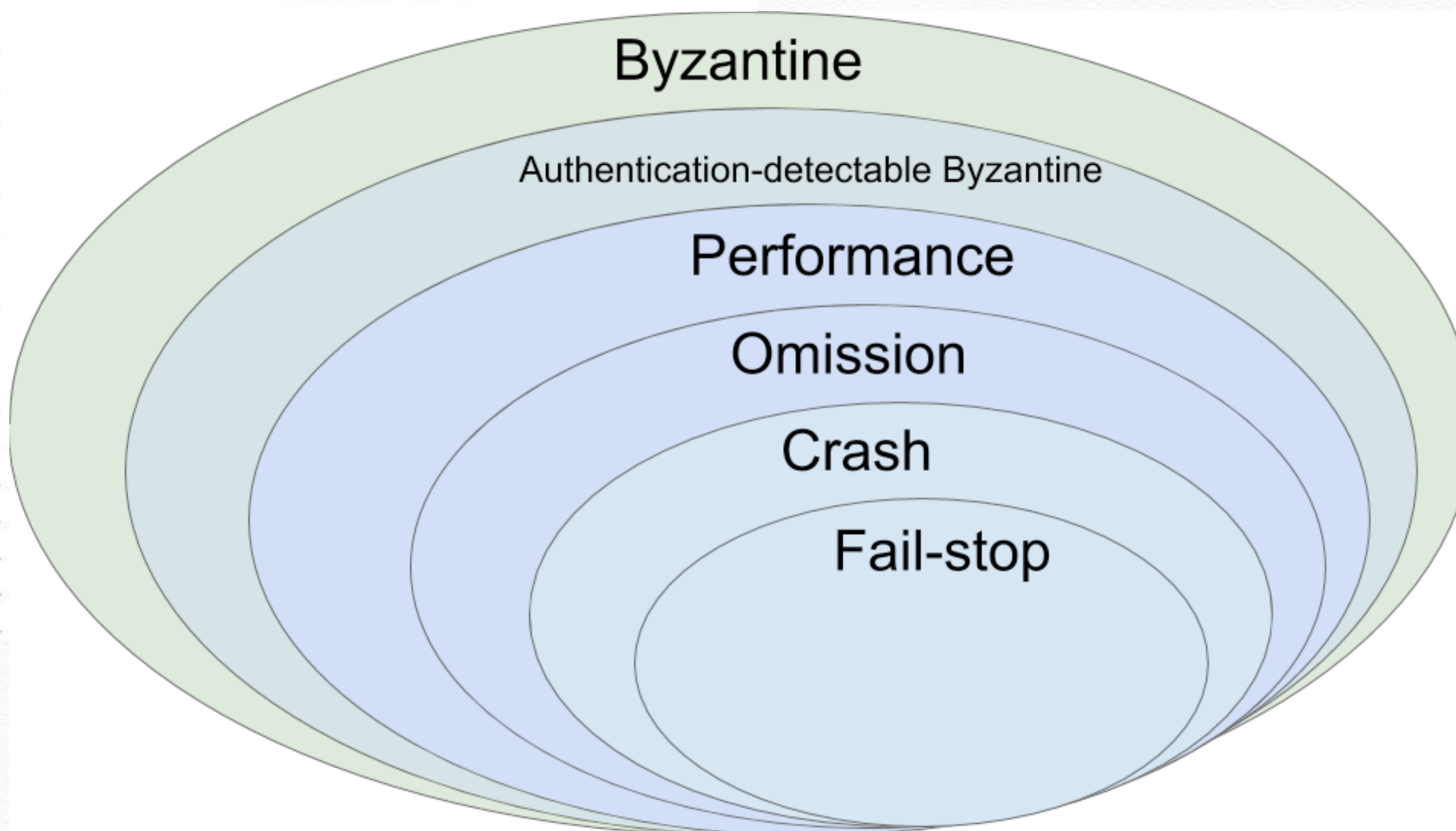
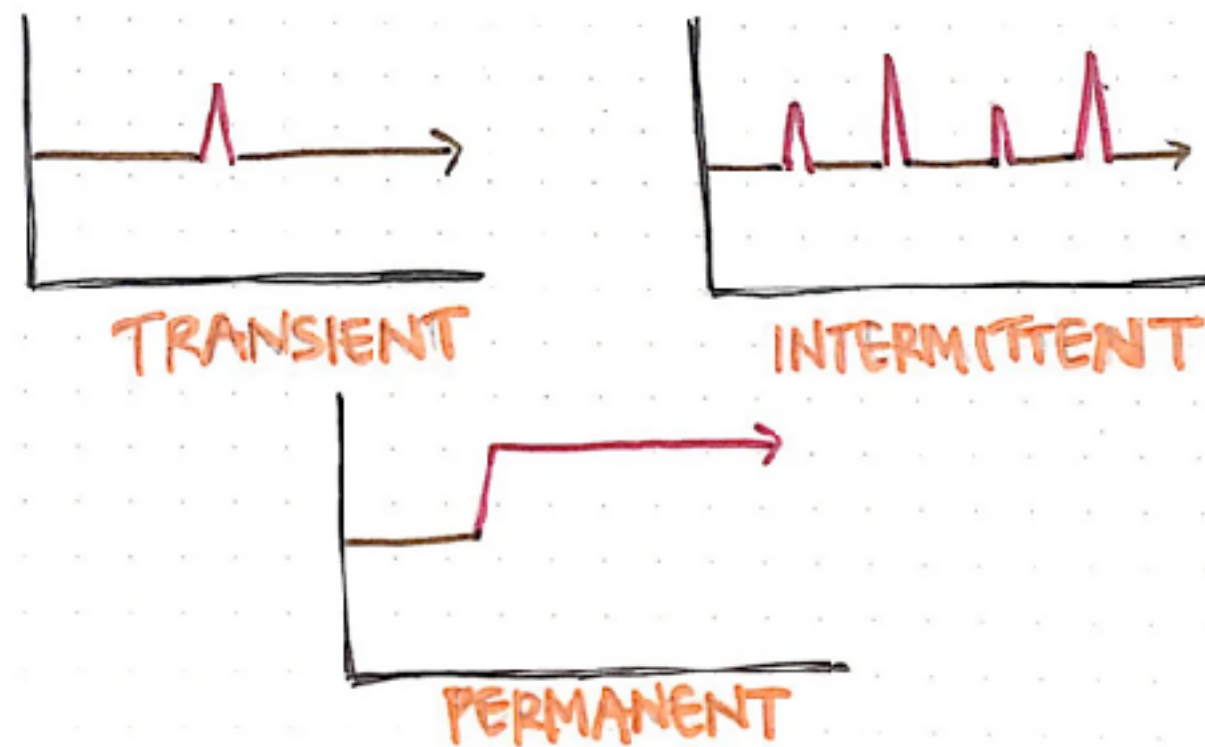


(A) Fail-silent/fail-stop:

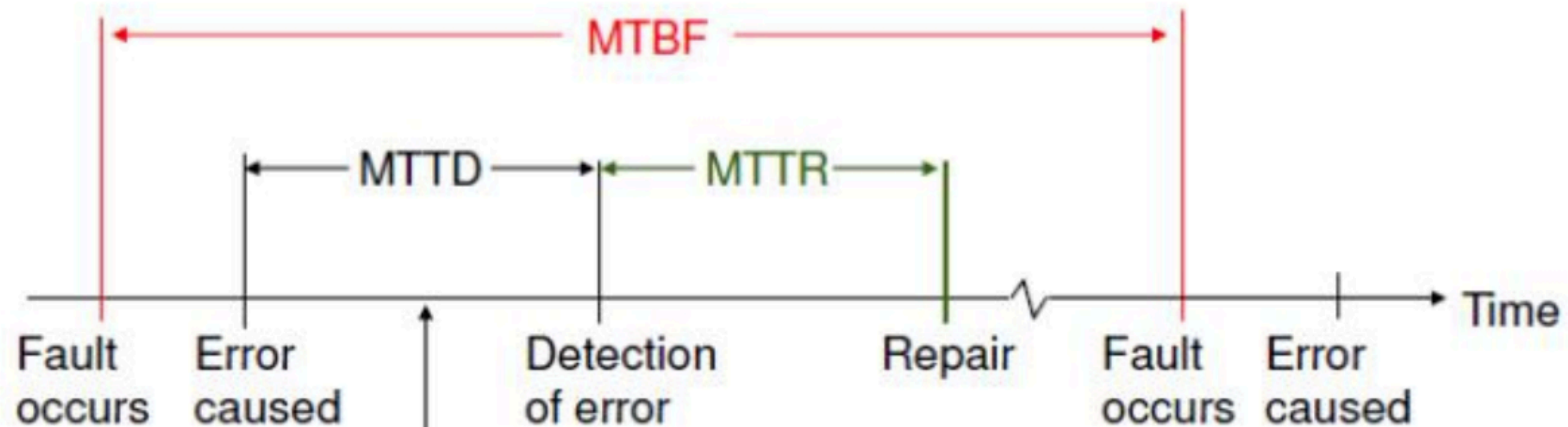
the node with the fault stops working, and either produces No result, or a result that shows the node failed.

(B) Byzantine failure:

the node with the fault may keep working, but produces an unexpected result/error.



Introdução



Multiple errors can occur during this period

MTBF: Mean Time Between Failure
MTTD: Mean Time To Discovery
MTTR: Mean Time to Repair

Redundância

- ❖ Mascarando falhas através da Redundância
 - ❖ Três tipos são possíveis: redundância da informação, redundância do tempo e redundância física.
 - ❖ **Redundância da informação** coloca bits extras para a posterior recuperação de bits invertidos. Por exemplo, código de hamming pode ser adicionado a uma seqüência de dados para a recuperação de ruído no meio de comunicação.
 - ❖ **Redundância de tempo** acontece quando uma ação é realizada e caso necessária, feita novamente. O uso de transações são um exemplo dessa redundância. Se uma transação aborta, ela deve ser refeita.
 - ❖ Importante: Redundância por tempo são especialmente importantes quando existem falhas transientes ou intermitentes.
 - ❖ **Redundância física** é expressada através de um equipamento extra ou de processo adicionais que fazem com que o sistema como um todo tolere a perda ou o mal funcionamentos de um ou vários de seus componentes. Assim, pode ser expressada tanto em hardware, quanto em software.
 - ❖ Importante: mamíferos possuem dois olhos, dois ouvidos, dois pulmões. Aviões normalmente possuem 4 motores, mas podem voar normalmente somente com 3.

Comunicação cliente-servidor confiável

- ❖ Em muitos sistemas, comunicação ponto-a-ponto confiável é estabelecida usando um protocolo de transporte confiável como o **TCP**. TCP mascara falhas de omissão (as quais ocorrem na forma de mensagens perdidas) usando ACK e retransmissões. Entretanto, falhas de crash nas conexões não são mascaradas. Uma falha desse tipo pode ocorrer abruptamente fechando uma conexão TCP. Na maioria dos casos, o cliente é informado através de uma exceção que o canal sofreu um crash.
- ❖ Quanto a sistemas com RPC, a idéia é fazer com que detalhes da comunicação sejam escondidos do programador. Podemos distinguir 5 tipos de falhas que podem ocorrer num RPC.
 - ❖ Cliente não consegue localizar o servidor
 - ❖ A mensagem de requisição do cliente para o servidor é perdida
 - ❖ O servidor sofre um crash depois de receber uma requisição
 - ❖ A mensagem de resposta de um servidor para um cliente é perdida
 - ❖ O cliente sofre um crash depois de enviar uma requisição
- ❖ **Falha 1 RPC: Cliente não consegue localizar o servidor** - O servidor está down ou o cliente pode estar usando um stub desatualizado. Podemos usar tratadores de exceção em Java e tratadores de sinais em C.
- ❖ **Falha 2 RPC: Mensagens de requisição são perdidas** - Mais fácil de lidar. Se um temporizador expirar antes que a resposta ou ACK retorne, a requisição é enviada novamente.

comunicação cliente-servidor confiável

- ❖ **Falha 3 RPC: Crash no servidor:** Observe a figura abaixo. Podemos esperar pelo reboot do servidor e tentar a operação novamente.

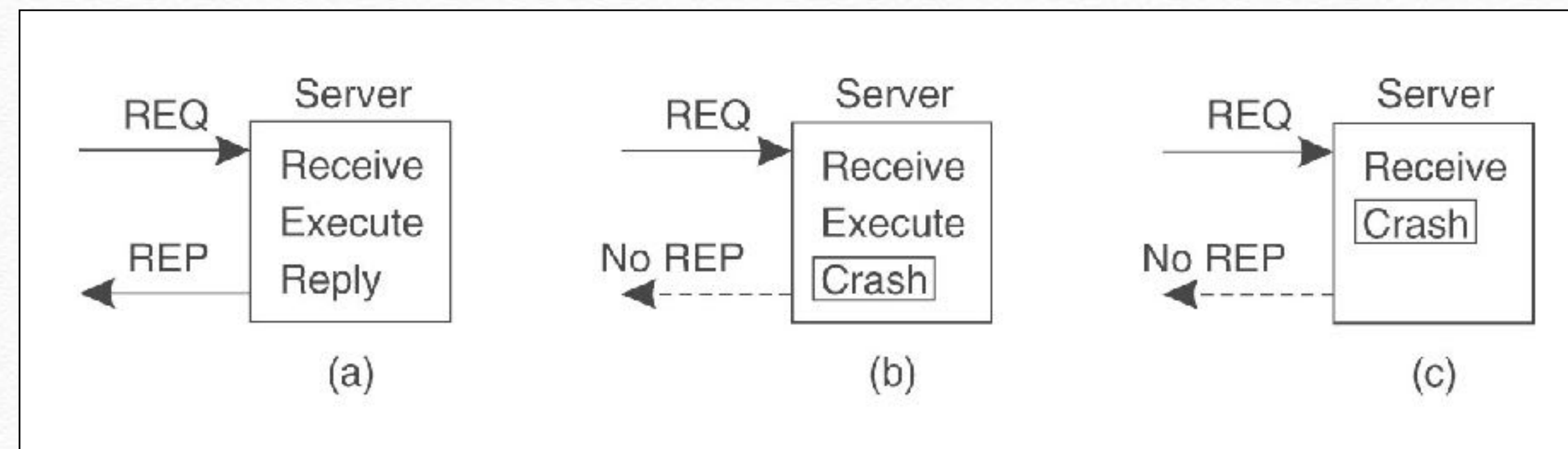


Figura 2.
Diferentes
situações na
falha de um
servidor em
interações RPC

- ❖ **Falha 4 RPC: Mensagens de resposta perdidas:** Esse tipo de falha é de difícil tratamento. Uma solução óbvia é simplesmente confiar num temporizador novamente que foi setado no sistema operacional do cliente.
 - ❖ Importante: Algumas operações podem ser repetidas tantas vezes quanto for necessário sem problemas. Exemplo: Capturar sempre os 1024 bytes de um determinado arquivo. Uma requisição com essa propriedade é chamada de **idempotente**. Entretanto, algumas operações, tipo as bancárias, não podem ser repetidas. Uma idéia é que cada cliente adicione na requisição um número de sequência. Assim, o servidor sabe todas as requisições que já processou com sucesso e não refaz a operação.

comunicação cliente-servidor confiável

❖ Falha 5 RPC: Cliente sofre um crash.

- ❖ O que acontece se um cliente envia uma requisição para um servidor e sofre um crash antes de receber a resposta? A computação está ativa e nenhum pai está esperando pelo resultado. Essa computação é denominada Órfão.
- ❖ Órfãos podem causar uma série de problemas. No mínimo, eles consomem ciclos de CPU. Eles podem fazer o lock de arquivos, o que pode significar a inutilização de um determinado recurso do sistema operacional. Em adição, caso o cliente faça um reboot e faça a requisição novamente, e caso o retorno do servidor acabe chegando, temos um problema para resolver.
- ❖ O que fazer com os Órfãos: exterminação, reencarnação e expiração.
 - ❖ Na segunda abordagem, o tempo é dividido numa seqüência de épocas. Quando um cliente recomeça, ele faz um broadcast de uma mensagem para todas as máquinas declarando que ele começou uma **nova época**. Todas as computações remotas que foram previamente lançadas por esse cliente são matadas. Obviamente, alguns órfãos podem ainda sobreviver. Quando eles são reportados ao cliente, eles terão uma época obsoleta e serão descartados.
 - ❖ Na terceira abordagem, cada **RPC recebe uma quantidade de tempo padrão T**. Se ela não conseguir terminar, deve ser feita a requisição por um novo *quantum*. Em outras palavras, se depois de um crash um cliente esperar por um tempo T antes de rebotar, temos a certeza que não existirão órfãos. O problema aqui é escolher um valor apropriado para T.

Recuperação - Recovery

- ❖ A principal idéia na recuperação de um erro é **substituir um estado de erro por outro livre de erros**. Basicamente, existem duas formas de recuperação de erros.
 - ❖ Backward Recovery: A principal questão é trazer o sistema de um estado com erros para outro anterior sem erros. Para isso, é necessário gravar o estado do sistema de tempos em tempos e recuperar esse estado gravado quando alguma coisa acontece de errado. Isso pode ser feito através de **checkpointing**.
 - ❖ Exemplo: Comunicação confiável através da retransmissão de pacotes.
 - ❖ Forward Recovery: Quando o sistema entrar num estado com erros, ao invés de se mover para trás em busca de um estado com checkpoint, uma tentativa é feita no sentido de tornar o sistema livre de erros através de um novo estado na continuação de sua execução.
- ★ ❖ Importante: O principal problema dessa abordagem é que devem ser conhecidos de antemão quais erros podem ocorrer.
- ❖ Ambas técnicas de recuperação podem ser integradas em middlewares para sistemas distribuídos. **Backward Recovery** introduz alguns problemas.
 - ❖ Primeiro, a restauração do sistema para um estado prévio é uma operação custosa em termos de desempenho.
 - ❖ Segundo, essa técnica é independente da aplicação distribuída, de modo que não há como saber o lugar que a recuperação irá ocorrer.

Recuperação - Recovery

❖ Checkpointing:

- ❖ A recuperação Backward requer que o sistema regularmente salve seu estado em um armazenamento estável.
- ❖ Num **snapshot distribuído**, se um processo P gravou o recebimento de uma mensagem, então deve existir um processo Q que tenha gravado o envio da mensagem.
- ❖ **Como funciona:** Cada processo salva seu estado de tempos em tempos localmente. Na recuperação, devemos construir um estado global consistente a partir das gravações locais. Para isso, devemos montar uma **linha de recuperação** que mostrará o estado consistente mais recente.

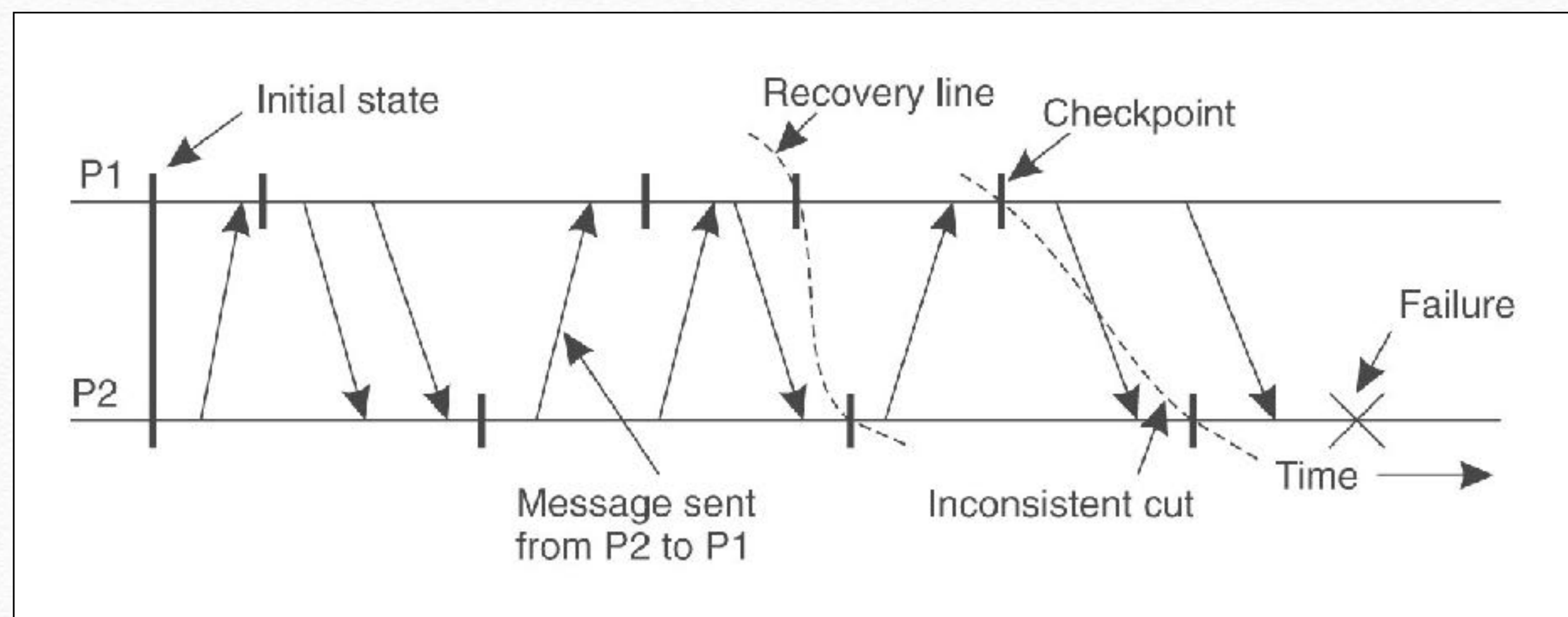


Figura 4. Linha de recuperação

Recuperação - Recovery

❖ Checkpointing independente:

- ❖ Naturalmente, o fato de o processo fazer checkpoint é uma tarefa local e sem coordenação com os demais.
- ❖ Para descobrir uma linha de recuperação é necessário que os processos voltem até o último estado gravado. Se todos os estados locais juntos não formam um snapshot distribuído, mais rolagens para trás são necessárias.
 - ❖ Importante: Esse processo pode levar a um **efeito dominó**.

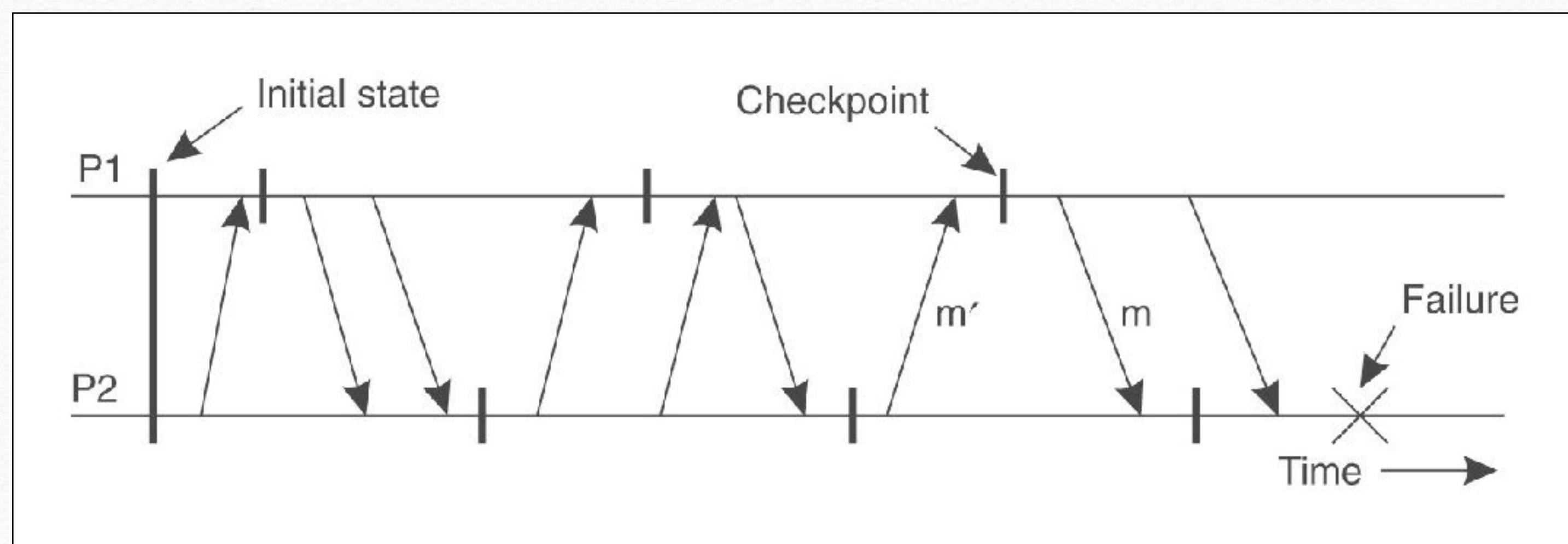


Figura 4.

Efeito

Dominó

Recuperação - Recovery

❖ Checkpoint independente (continuação)

- ❖ Por fim, o cálculo de linhas de recuperação requer uma análise do intervalo de dependência armazenado em cada processo.
- ❖ Importante: Checkpoint independente é um tratamento complexo para a Backward recovery e a tomada de checkpoint coordenada pode ser usada nesse contexto. Não é a coordenação entre os processos que é o fator dominante de desempenho, mas sim a sobrecarga como resultado de ter que gravar o estado no armazenamento estável local.
 - ❖ Assim, **checkpoint coordenado** é uma abordagem mais popular e corriqueiramente usada em middlewares que implementam Recovery.

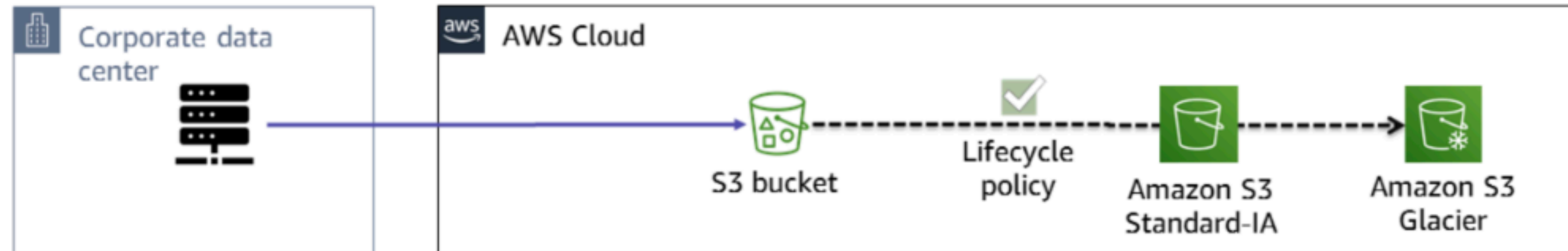
❖ Checkpoint Coordenado

- ❖ Todos os processos se sincronizam para juntamente escreverem o seus estados no armazenamento estável local. A principal vantagem é que **o estado salvo automaticamente consistente globalmente.**
 - ❖ Importante: Essa abordagem evita de roollback sucessivos gerado pelo efeito dominó
- ❖ Como fazer: Snapshot distribuído pode ser usado para a realização do checkpoint coordenado. A implementação usada no Snapshot distribuído é não bloqueante.

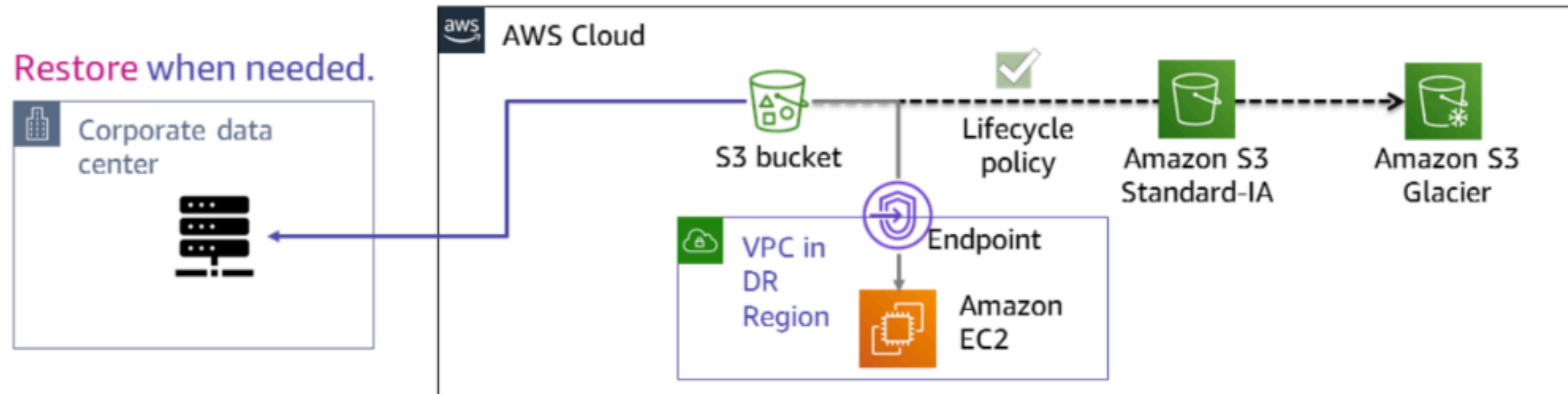
Cloud computing: Backup e Restore

Backup and restore pattern

Back up configuration and state data to S3. Implement lifecycle policy to save on cost.



Restore when needed.



Cloud computing: Backup e Restore

Backup and restore: Checklist

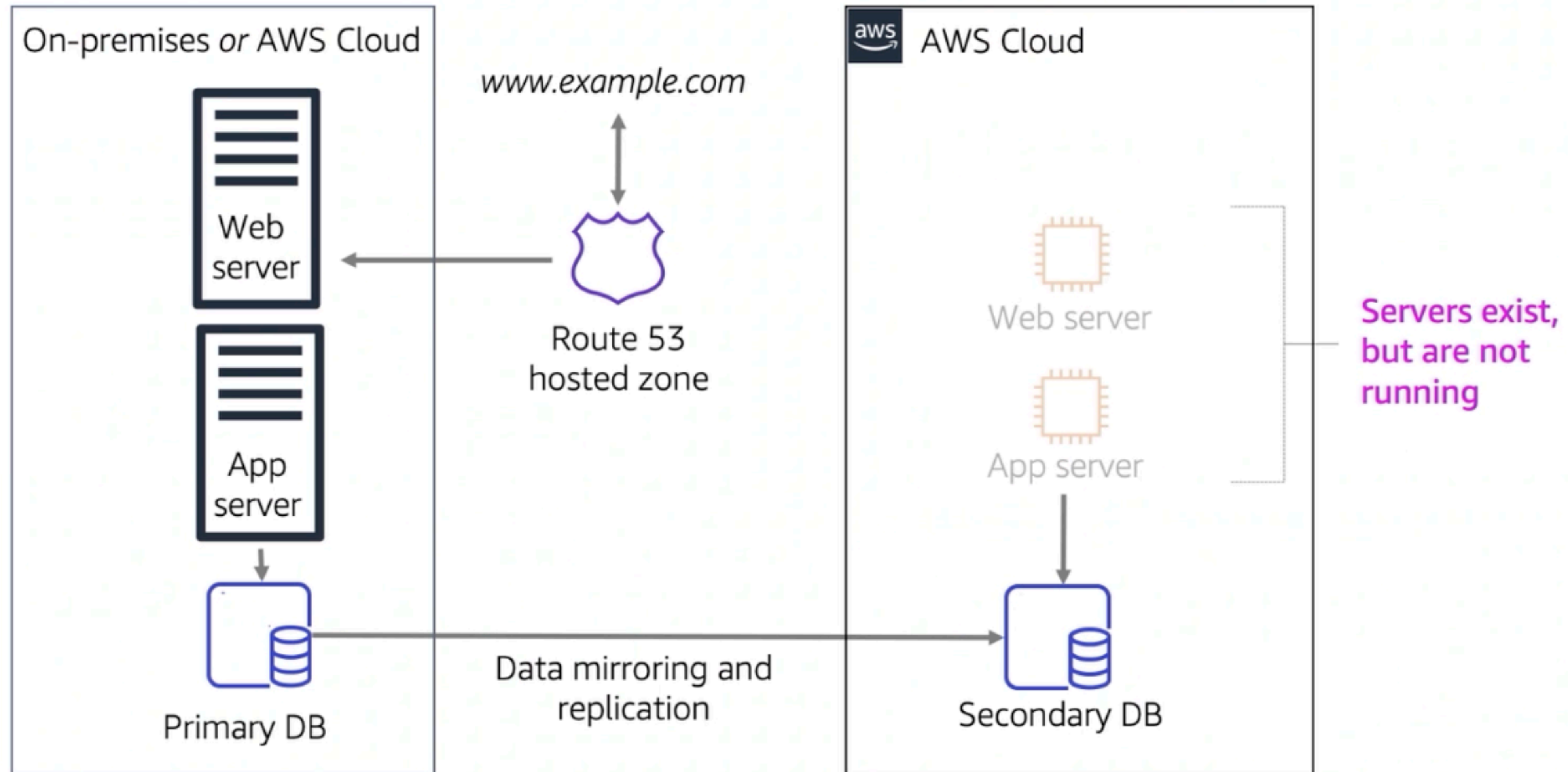
Preparation phase

- Create backups of current systems
- Store backups in Amazon S3
- Document procedure to restore from backups
- Know:
 - Which AMI to use, and build as needed
 - How to restore system from backups
 - How to route traffic to the new system
 - How to configure the deployment

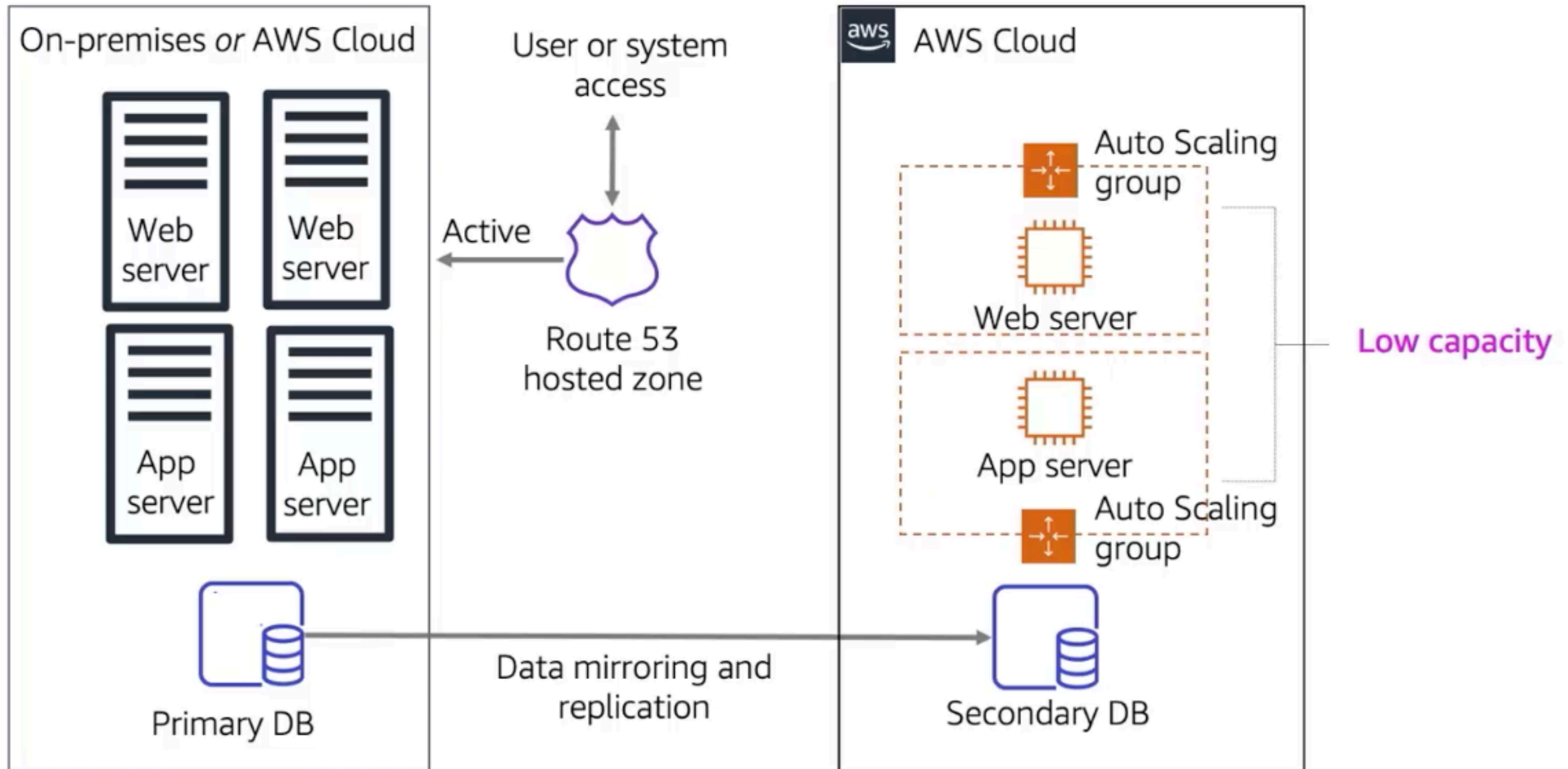
In case of disaster

- Retrieve backups from Amazon S3
- Restore required infrastructure
 - EC2 instances from prepared AMIs
 - Elastic Load Balancing load balancers
 - AWS resources created by an AWS CloudFormation stack – automated deployment to restore or duplicate the environment
- Restore system from backup
- Route traffic to the new system
 - Adjust Domain Name System (DNS) records accordingly

Disaster Recovery em cloud: Alternativa 1



Disaster Recovery em cloud: Alternativa 2



Disaster Recovery em cloud: Alternativa 3

